

100 задач по слайсам в Go

1.

```
func Sum(s []int) int
```

Верни сумму всех элементов слайса. Для пустого слайса верни 0.

2.

```
func Min(s []int) (int, bool)
```

Верни минимальный элемент и true. Для пустого слайса верни 0 и false.

3.

```
func Max(s []int) (int, bool)
```

Верни максимальный элемент и true. Для пустого слайса верни 0 и false.

4.

```
func CountEven(s []int) int
```

Посчитай количество чётных элементов.

5.

```
func Contains(s []int, target int) bool
```

Верни true, если target встречается в слайсе хотя бы один раз.

6.

```
func CountOccurrences(s []int, target int) int
```

Посчитай, сколько раз target встречается в слайсе.

7.

```
func IndexOf(s []int, target int) int
```

Верни индекс первого вхождения target. Если элемента нет, верни -1.

8.

```
func IndexesOf(s []int, target int) []int
```

Верни слайс всех индексов, на которых встречается target, в возрастающем порядке.

9.

```
func ReplaceAll(s []int, old, new int)
```

Замени все значения old на new прямо в исходном слайсе. Новый слайс создавать нельзя.

10.

```
func Reverse(s []int)
```

Разверни слайс in-place. Дополнительный слайс создавать нельзя; O(1) дополнительной памяти.

11.

```
func SwapPairs(s []int)
```

Поменяй местами соседние элементы: [1,2,3,4,5] -> [2,1,4,3,5]. Работай in-place.

12.

```
func ReverseRange(s []int, left, right int) error
```

Разверни in-place участок с индексами от left до right включительно. Некорректные границы должны возвращать ошибку, не меняя слайс.

13.

```
func RemoveAt(s []int, i int) ([]int, error)
```

Удали элемент с индексом i, сохранив порядок остальных элементов. Используй существующий backing array, если это возможно.

14.

```
func RemoveAtFast(s []int, i int) ([]int, error)
```

Удали элемент с индексом i за O(1), если порядок остальных элементов сохранять не требуется.

15.

```
func RemoveAll(s []int, target int) []int
```

Верни новый независимый слайс без элементов, равных target. Результат не должен alias-ить входной слайс.

16.

```
func RemoveAllInPlace(s []int, target int) []int
```

Удали все target in-place, сохранив порядок. Новый backing array выделять нельзя.

17.

```
func KeepEvenInPlace(s []int) []int
```

Оставь только чётные элементы, сохранив их порядок. Работай in-place без нового backing array.

18.

```
func MoveZerosToEnd(s []int)
```

Перемести все нули в конец, сохранив относительный порядок ненулевых элементов. Работай in-place и O(1) дополнительной памяти.

19.

```
func CompactNonZero(s []int) []int
```

Сдвинь ненулевые элементы в начало слайса, сохрани порядок, и верни слайс только с ними. Хвост результата должен быть исключён через изменение len.

20.

```
func InsertAt(s []int, i, value int) ([]int, error)
```

Вставь value перед элементом с индексом i; i может быть равен len(s). Сохрани порядок. Используй append/copy; корректно работай и при realloc.

21.

```
func InsertMany(s []int, i int, values []int) ([]int, error)
```

Вставь весь values перед индексом i. values может быть пустым. Сохрани порядок элементов.

22.

```
func DeleteRange(s []int, left, right int) ([]int, error)
```

Удали полуинтервал [left:right), сохранив порядок. Для пустого диапазона верни исходный слайс без изменений.

23.

```
func ReplaceRange(s []int, left, right int, values []int) ([]int, error)
```

Замени полуинтервал [left:right) элементами values. Количество удаляемых и вставляемых элементов может различаться.

24.

```
func Splice(s []int, start, deleteCount int, values []int) ([]int, error)
```

Реализуй аналог splice: начиная со start удали deleteCount элементов и вставь values. Сохрани порядок и минимизируй лишние аллокации.

25.

```
func Clone(s []int) []int
```

Верни полную независимую копию слайса. Изменения результата не должны влиять на исходный слайс и наоборот.

26.

```
func CloneRange(s []int, left, right int) ([]int, error)
```

Верни независимую копию полуинтервала [left:right). Не возвращай subslice исходного backing array.

27.

```
func Concat(a, b []int) []int
```

Объедини два слайса в новый независимый слайс. Постарайся сделать не более одной аллокации под backing array результата.

28.

```
func ConcatMany(parts ...[]int) []int
```

Объедини произвольное количество слайсов в один независимый слайс. Сначала вычисли итоговую длину и постарайся выделить память один раз.

29.

```
func Unique(s []int) []int
```

Верни уникальные элементы в порядке первого появления. Результат должен быть независимым от исходного слайса.

30.

```
func UniqueInPlace(s []int) []int
```

Удаляй повторы in-place, сохраняя первое появление каждого значения и исходный порядок. Допускается map для проверки наличия, но новый backing array для результата создавать нельзя.

31.

```
func UniqueSortedInPlace(s []int) []int
```

Для отсортированного слайса удали дубликаты in-place за $O(n)$ времени и $O(1)$ дополнительной памяти.

32.

```
func KeepAtMostKDuplicates(s []int, k int) ([]int, error)
```

Слайс отсортирован. Оставь не более k копий каждого значения, работая in-place за $O(n)$.

33.

```
func IntersectionUnique(a, b []int) []int
```

Верни уникальные значения, которые встречаются и в a , и в b . Порядок результата должен соответствовать первому появлению в a .

34.

```
func IntersectionMulti(a, b []int) []int
```

Верни пересечение с учётом количества повторений: каждое значение должно встретиться $\min(\text{countA}, \text{countB})$ раз.

35.

```
func Difference(a, b []int) []int
```

Верни элементы a , значения которых вообще не встречаются в b . Сохрани порядок a .

36.

```
func SymmetricDifference(a, b []int) []int
```

Верни значения, присутствующие ровно в одном из двух слайсов. Каждый результат должен встречаться один раз.

37.

```
func MergeSorted(a, b []int) []int
```

Объедини два отсортированных по возрастанию слайса в один отсортированный за $O(\text{len}(a) + \text{len}(b))$.

38.

```
func MergeSortedUnique(a, b []int) []int
```

Объедини два отсортированных слайса, одновременно удаляя дубликаты, за линейное время.

39.

```
func IsSorted(s []int) bool
```

Проверь, отсортирован ли слайс по неубыванию.

40.

```
func Partition(s []int, pivot int) int
```

Переставь элементы in-place так, чтобы сначала шли элементы $< \text{pivot}$, затем элементы $\geq \text{pivot}$. Порядок сохранять не требуется. Верни индекс начала второй части.

41.

```
func StablePartition(s []int, pivot int) []int
```

Переставь элементы так, чтобы сначала шли $< \text{pivot}$, затем $\geq \text{pivot}$, сохранив относительный порядок внутри обеих групп. Дополнительный слайс разрешён.

42.

```
func StablePartitionInPlace(s []int, pivot int)
```

То же самое, но новый backing array создавать нельзя. Дополнительная память $O(1)$. Разрешена сложность хуже $O(n)$, но решение должно быть корректным и устойчивым.

43.

```
func RotateLeft(s []int, k int)
```

Циклически сдвинь слайс влево на k позиций in-place. k может быть больше $\text{len}(s)$ и может быть отрицательным.

44.

```
func RotateRight(s []int, k int)
```

Циклически сдвинь слайс вправо на k позиций in-place, используя $O(1)$ дополнительной памяти.

45.

```
func NextPermutation(s []int) bool
```

Преобразуй слайс в следующую лексикографическую перестановку in-place. Верни false и преврати его в минимальную перестановку, если следующей не существует.

46.

```
func DutchFlag(s []int, pivot int)
```

Переставь элементы in-place за один проход так, чтобы сначала шли $< \text{pivot}$, затем $== \text{pivot}$, затем $> \text{pivot}$. Порядок сохранять не требуется.

47.

```
func ChunkViews(s []int, size int) ([][]int, error)
```

Разбей слайс на чанки размера size . Последний может быть короче. Чанки должны быть views одного backing array, без копирования данных.

48.

```
func ChunkCopies(s []int, size int) ([][]int, error)
```

Разбей на чанки того же размера, но каждый чанк должен иметь независимый backing array.

49.

```
func SplitIntoKViews(s []int, k int) ([][]int, error)
```

Разбей слайс на k непустых или пустых views максимально равного размера: длины частей должны отличаться не более чем на 1.

50.

```
func SlidingWindowViews(s []int, size int) ([][]int, error)
```

Верни все подряд идущие окна длины size как subslice исходного backing array. Для $\text{size} > \text{len}(s)$ верни пустой результат.

51.

```
func SlidingWindowCopies(s []int, size int) ([][]int, error)
```

Верни те же окна, но каждое окно должно быть независимой копией.

52.

```
func SplitOnValue(s []int, sep int) [][]int
```

Разбей слайс на части по значениям `sep`, аналогично `strings.Split`. Определи и последовательно обработай пустые части между соседними `sep` и по краям.

53.

```
func TrimZeros(s []int) []int
```

Верни `slice` без ведущих и хвостовых нулей. Не копируй данные и не меняй `backing array`.

54.

```
func ExpandToCap(s []int) []int
```

Верни `reslice`, у которого `len == cap` исходного `s`. Данные не копируй. Для `nil slice` результат должен оставаться `nil`.

55.

```
func ClipCapacity(s []int) []int
```

Верни `slice view` с теми же элементами, но `cap` результата должен быть равно `len(s)`. Новый `backing array` создавать нельзя.

56.

```
func TakeWithLimitedCap(s []int, n int) ([]int, error)
```

Верни первые `n` элементов как `view`, причём `cap` результата должен быть равно `n`, чтобы последующий `append` не мог затереть элементы `s` после `n`.

57.

```
func Resize(s []int, n int) ([]int, error)
```

Измени длину до `n`. При увеличении используй существующий `cap` без аллокации, если его хватает; новые элементы должны быть нулевыми. При уменьшении верни `reslice`.

58.

```
func EnsureCapacity(s []int, minCap int) ([]int, error)
```

Гарантируй `cap >= minCap`, сохранив `len` и значения. Если `cap` уже хватает, верни `slice` на том же `backing array`; иначе выдели новый.

59.

```
func TryAppend(s []int, value int) ([]int, bool)
```

Добавь `value` только если это можно сделать без `realloc`. Если `cap` не хватает, верни исходный `slice` и `false`, не меняя `backing array`.

60.

```
func AppendCopy(s []int, values ...int) []int
```

Верни результат `append`, который гарантированно не `alias`-ит исходный `backing array s`, даже если в `s` было достаточно свободного `cap`.

61.

```
func CopyWithin(s []int, dst, left, right int) error
```

Скопируй участок `[left:right)` начиная с индекса `dst` внутри того же слайса. Корректно обрабатывай перекрывающиеся диапазоны и проверяй границы.

62.

```
func InsertWithCopy(s []int, i int, values []int) ([]int, error)
```

Реализуй вставку `values` через увеличение длины и `copy`. `values` может `alias`-ить `s`; функция должна оставаться корректной.

63.

```
func DeleteAndClear[T any](s []T, left, right int) ([]T, error)
```

Удаляй `[left:right)` in-place, сохраняя порядок, а освободившийся хвост `backing array` обнуляй через `clear`, чтобы не удерживать ссылки на удалённые объекты.

64.

```
func CompactAndClear[T any](s []T, keep func(T) bool) []T
```

Стабильно отфильтруй слайс in-place по keep, а хвост backing array после новой длины очисти через clear.

65.

```
func AppendValue(s []int, value int) []int
```

Напиши функцию-обёртку над append с правильным API: вызывающая сторона должна иметь возможность получить новый slice header после возможного realloc.

66.

```
func AppendInPlace(s *[]int, value int)
```

Добавь value, изменяя сам slice header вызывающей стороны через указатель на слайс.

67.

```
func DropFirst(s []int, n int) ([]int, error)
```

Верни view без первых n элементов. Не копируй данные. Корректно обработай n от 0 до len(s).

68.

```
func DropFirstInPlace(s *[]int, n int) error
```

Измени slice header вызывающей стороны так, чтобы он больше не включал первые n элементов.

69.

```
func DeleteAtInPlace(s *[]int, i int) error
```

Удаляй элемент по индексу с сохранением порядка и записывай новый slice header обратно через *[]int.

70.

```
func AppendToSubslice(base []int, left, right int, values ...int) ([]int, error)
```

Возьми view base[left:right] и добавь values. Функция должна гарантировать, что элементы base за right не будут изменены, даже если исходного cap достаточно.

71.

```
func SafeSubslice(s []int, left, right int) ([]int, error)
```

Верни view [left:right] с ограниченным cap == len(result), чтобы caller не мог append-ом затереть соседние элементы исходного backing array.

72.

```
func DetachIfShared(s []int) []int
```

Верни независимую копию с теми же элементами и cap == len(s). Функция всегда должна разрывать aliasing с исходным backing array.

73.

```
func RowSums(m [][]int) []int
```

Верни сумму каждой строки рванной матрицы.

74.

```
func ColumnSums(m [][]int) []int
```

Для рванной матрицы верни суммы по столбцам до максимальной длины строки; отсутствующие элементы не учитывай.

75.

```
func Flatten(m [][]int) []int
```

Слей все строки в один слайс в порядке обхода строк. Постарайся заранее вычислить итоговую длину и сделать одну аллокацию.

76.

```
func Unflatten(s []int, sizes []int) ([][]int, error)
```

Разбей s на последовательные независимые части заданных размеров. Сумма sizes должна равняться len(s), иначе ошибка.

77.

```
func Transpose(m [][]int) ([][]int, error)
```

Транспонируй прямоугольную матрицу. Если строки имеют разные длины, верни ошибку.

78.

```
func TransposeRagged(m [][]int) [][]int
```

Транспонируй рваную матрицу: результат должен иметь столько строк, какова максимальная длина исходной строки; пропущенные элементы просто не добавляй.

79.

```
func RemoveColumn(m [][]int, col int) [][]int
```

Удаляй столбец col из каждой строки, где он существует. Сохраняй порядок элементов строк.

80.

```
func RotateSquare90(m [][]int) error
```

Поверни квадратную матрицу на 90 градусов по часовой стрелке in-place. Если матрица не квадратная, верни ошибку без частичного изменения.

81.

```
func SpiralOrder(m [][]int) ([]int, error)
```

Верни элементы прямоугольной матрицы по спирали. Для рванной матрицы верни ошибку.

82.

```
func PrefixSums(s []int) []int
```

Построй новый слайс prefix длины len(s)+1, где prefix[0]=0, а prefix[i+1] - сумма первых i+1 элементов.

83.

```
func ProductExceptSelf(s []int) []int
```

Для каждого индекса верни произведение всех остальных элементов без деления. O(n) времени; не используй O(n) дополнительной памяти кроме результата.

84.

```
func TwoSumSorted(s []int, target int) (i, j int, ok bool)
```

Слайс отсортирован. Найди два разных индекса с суммой target за O(n) и O(1) памяти.

85.

```
func ThreeSum(s []int) [][]int
```

Верни все уникальные тройки значений с суммой 0. Дубликатов троек быть не должно. Разрешается сортировать копию входного слайса.

86.

```
func LongestIncreasingRun(s []int) (start, end int)
```

Найди самый длинный непрерывный строго возрастающий участок и верни его как полуинтервал [start:end). При равенстве длин выбери самый левый.

87.

```
func MinSubarrayLenAtLeast(s []int, target int) int
```

Все элементы s неотрицательные. Верни минимальную длину непрерывного подмассива с суммой $\geq$ target или 0, если такого нет. Требуется O(n).

88.

```
func LongestSubarrayAtMostKDistinct(s []int, k int) (start, end int)
```

Найди самый длинный непрерывный участок, содержащий не более k различных значений. Требуется O(n) в среднем.

89.

```
func SlidingWindowMax(s []int, k int) ([]int, error)
```

Верни максимум каждого окна длины k за O(n). Используй deque, реализованный на slice, без container/list.

90.

```
func MergeIntervals(intervals [][]int) ([][]int, error)
```

Каждый интервал имеет вид [start,end]. Объедини пересекающиеся интервалы и верни непересекающийся отсортированный результат. Некорректные интервалы должны давать ошибку.

91.

```
func InsertInterval(intervals [][]int, newInterval []int) ([][]int, error)
```

intervals уже отсортированы и не пересекаются. Вставь новый интервал и слей необходимые пересечения за $O(n)$.

92.

```
func MergeIntoCapacity(a []int, aLen int, b []int) ([]int, error)
```

Первые aLen элементов a отсортированы, len(a)==aLen, но cap(a) >= aLen+len(b). b тоже отсортирован. Объедини b в a in-place, не выделяя новый backing array, и верни расширенный slice.

93.

```
func MergeSortedInPlace(a []int, b []int) ([]int, error)
```

a и b - соседние отсортированные части одного backing array: a заканчивается там, где начинается b. Слей их в один отсортированный диапазон без дополнительного backing array. Допускается сложность $O(n*m)$, но решение должно корректно работать при aliasing.

94.

```
func RotateByCycles(s []int, k int)
```

Сделай циклический сдвиг вправо на k in-place через циклы перестановки (алгоритм с gcd), не используя тройной reverse и второй слайс.

95.

```
func Permutations(s []int) [][]int
```

Верни все перестановки элементов. Каждый элемент результата должен иметь независимый backing array; никакие две перестановки не должны alias-ить друг друга.

96.

```
func ContiguousViews(s []int) [][]int
```

Верни все непустые непрерывные subslice исходного s как views без копирования. Порядок: сначала по start, затем по end.

97.

```
func ContiguousCopies(s []int) [][]int
```

Верни все непустые непрерывные подмассивы как независимые копии. Ни один внутренний slice не должен alias-ить s или другой результат.

98.

```
func SplitBalancedViews(s []int, k int) ([][]int, error)
```

Раздели s на k последовательных views максимально равного размера. Дополнительно ограничь cap каждой части её len, чтобы append к одной части не мог затереть соседнюю.

99.

```
type Edit struct { Start, Delete int; Insert []int }
func ApplyEdits(s []int, edits []Edit) ([]int, error)
```

edits отсортированы по Start, не пересекаются и задаются относительно исходного s. За один проход примени все удаления и вставки, сохраняя порядок. Постарайся заранее вычислить конечную длину и сделать не более одной новой аллокации.

100.

```
func SpliceAliasingSafe(s []int, start, deleteCount int, insert []int) ([]int, error)
```

Реализуй splice, причём insert может быть subslice самого s и пересекаться как с удаляемой областью, так и с областью, которая будет сдвигаться. Результат должен быть корректным при любом таком aliasing и по возможности переиспользовать backing array.